

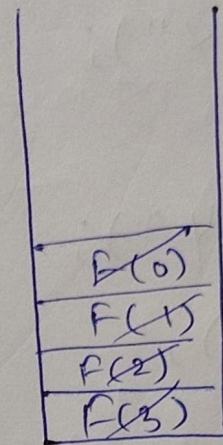
Assignment No - 6.

Topic : Recursion Dry Run - Flow of parameter changes.

Q.1] predict the output.

```
void fun(int n){
    if (n==0) return;
    s.o.p ( n + " " );
    F(n-1);
    S.o.p ((n+1) + " ");
}
```

Call: fun(3);



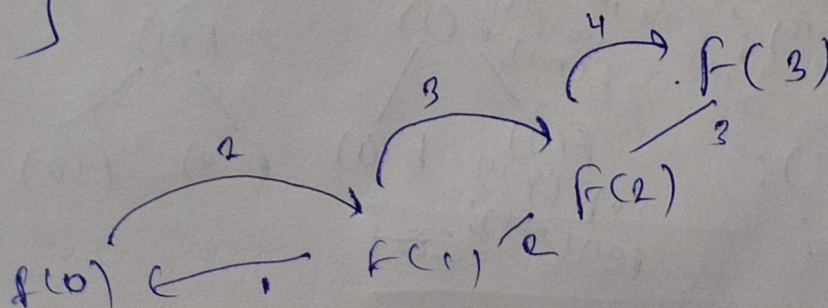
O/P:
3
2
1
2
3
4

³
void fun(int n){
 if (n==0) return;
 S.o.p (n + " ");
 F(n-1); ₃₋₁₌₂
 S.o.p((n+1) + " ");
}

²
void fun(int n){
 if (n==0) return;
 S.o.p (n + " ");
 F(n-1); ₂₋₁₌₁
 S.o.p ((n+1) + " ");
}

¹
void fun(int n){
 if (n==0) return;
 S.o.p (n + " ");
 F(n-1); ₁₋₁₌₀
 S.o.p ((n+1) + " ");
}

⁰
void fun(int n){
 if (n==0) return;
 S.o.p (n + " ");
 F(n-1); ₀₋₁₌₋₁
 S.o.p ((n+1) + " ");
}



Q.2) predict the output.

```
void fun (int2 n) {
    if (n == 0) return;
    s.o.p ("x");
    fun(n-1);
    s.o.p ("y");
    fun(n-1);
}
```

Total = 7 call.

	O/p
F(0)	X
F(0)	X
F(1)	Y
F(0)	Y
F(0)	Y
F(1)	X
F(2)	Y

fun(2);

²
void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1); 2-1=1
 s.o.p ("y");
 fun(n-1);
}

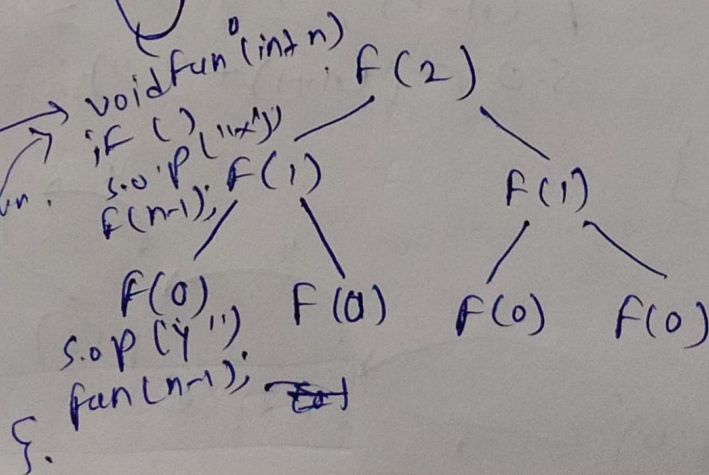
¹
void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1); 1-1=0
 s.o.p ("y");
 fun(n-1);
}

⁰
void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1);
 s.o.p ("y");
 fun(n-1);
}

void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1);
 s.o.p ("y");
 fun(n-1); 1-1=0
}

⁰
void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1);
 s.o.p ("y");
 fun(n-1); 0-1
}

⁰
void fun (int n) {
 if (n == 0) return;
 s.o.p ("x");
 fun(n-1);
 s.o.p ("y");
 fun(n-1);
}



Q.3)

```

4
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }

```

```

    fun(n-2); 4-4=2
    S.o.p(n + " ");
    fun(n-2);
}

```

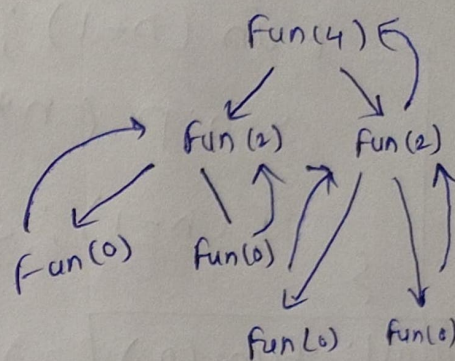
call: fun(4);

F(0)
F(0)
F(2)
F(0)
F(0)
F(2)
F(4)

O/P

0
2
0
4
0
2
0

Total - 7 call



Tree

```

2
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }

```

```

    fun(n-2); 2-2=0
    S.o.p(n + " ");
    fun(n-2);
}

```

```

0
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }
    fun(n-2);
    S.o.p(n + " ");
    fun(n-2);
}

```

```

0
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }
    fun(n-2);
    S.o.p(n + " ");
    fun(n-2);
}

```

```

2
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }

```

```

    fun(n-2);
    S.o.p(n + " ");
    fun(n-2);
}

```

```

0
void fun(int n){
    if (n <= 1){
        S.o.p(n + " ");
        return;
    }
    fun(n-2);
    S.o.p(n + " ");
    fun(n-2);
}

```

```

    fun(n-2);
    S.o.p(n + " ");
    fun(n-2);
}

```



```

void fun(int n){
    if (n==0) return;

```

```

    fun(n-1);
    s.o.p (n*n + " ");

```

```

    f(n-1);
}

```

Call: fun(2)

```

void fun(int n){
    if (n==0) return;

```

```

    fun(n-1);
    s.o.p (n*n + " ");
    f(n-1);
}

```

```

void fun(int n){
    if (n==0) return;

```

```

    fun(n-1);
    s.o.p (n*n + " ");
    f(n-1);
}

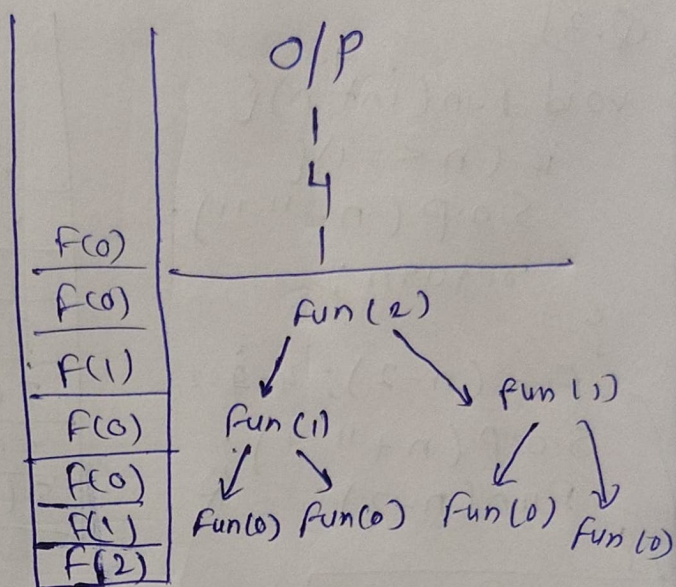
```

```

void fun(int n){
    if (n==0) return;
    f(n-1);
    s.o.p (n*n + " ");
    f(n-1)
}

```

Total call: 7



```

void fun(int n){
    if (n==0) return;
    fun(n-1);
    s.o.p (n*n + " ");
    f(n-1);
}

```

```

void fun(int n){
    if (n==0) return;
    f(n-1);
    s.o.p (n*n + " ");
    f(n-1);
}

```

```

void fun(int n){
    if (n==0) return;
    f(n-1);
    s.o.p (n*n + " ");
    f(n-1);
}

```


Q.5) ⁵void fun(int n){
 if (n <= 0) return;
 s.o.p (n + " ");

$F(n-2);$ $5-2=3$
 s.o.p (n + " ");

}

Cell: $F(5);$

³void fun (int n){
 if (n <= 0) return;
 s.o.p (n + " ");

$F(n-2);$ $3-2=1$
 s.o.p (n + " ");

};

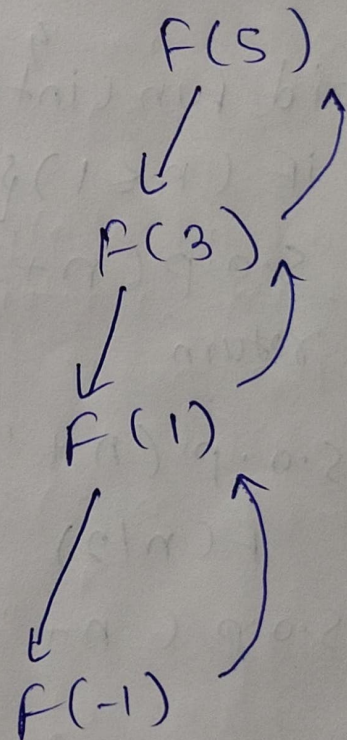
¹void fun (int n){
 if (n <= 0) return;
 s.o.p (n + " ");

$F(n-2);$ $1-2=-1$
 s.o.p (n + " ");
 };

⁻¹void fun (int n){
 if (n <= 0) return;
 s.o.p (n + " ");
 F (n-2);
 s.o.p (n + " ");
 }
 }

$F(5)$
$F(3)$
$F(1)$
$F(-1)$

o/p
 5
 3
 1
 1
 3
 5



Q.6]

```
void fun (int n) {
    if (n ≤ 1) {
        S.o.p (n + " ");
        return;
    }
```

```
    S.o.p (n + " ");
```

```
    f(n/2);  $8/2 = 4$ 
```

```
    S.o.p (n + " ");
}
```

Call: fun(8);

```
void fun (int n) {
    if (n ≤ 1) {
        S.o.p (n + " ");
        return;
    }
```

```
    S.o.p (n + " ");
```

```
    f(n/2)  $4/2 = 2$ 
```

```
    S.o.p (n + " ");
}
```

```
void fun (int n) {
    if (n ≤ 1) {
        S.o.p (n + " ");
        return;
    }
```

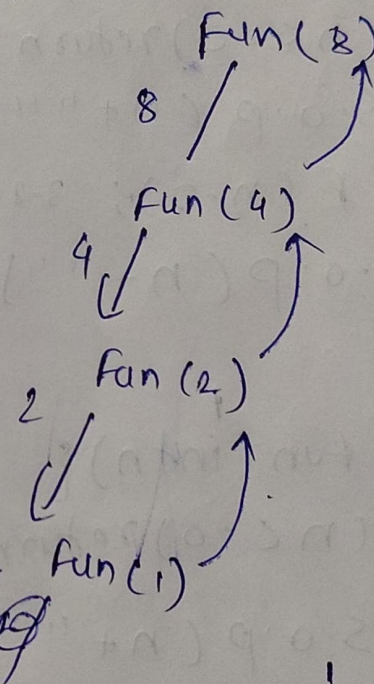
```
    S.o.p (n + " ");
```

```
    f(n/2)  $2/2 = 1$ 
```

```
    S.o.p (n + " ");
}
```

;

	O/P
	8
	4
	2
	1
f(1)	2
f(2)	4
f(4)	8
f(8)	



```
void fun (int n) {
    if (n ≤ 1) {
        S.o.p (n + " ");
        return;
    }
```

```
    S.o.p (n + " ");
```

```
    f(n/2);
```

```
    S.o.p (n + " ");
}
```

;

Q.7) void fun (int n) {
 if (n <= 0) return;
 S.o.p ("A" + n + " ");

Fun (n-1);

S.o.p ("B" + (n*2) + " ");

call: fun(3);

void fun (int n) {

if (n <= 0) return;

S.o.p ("A" + n + " ");

fun (n-1);

S.o.p ("B" + (n*2) + " ");

}

void fun (int n) {

if (n <= 0) return;

S.o.p ("A" + n + " ");

fun (n-1);

S.o.p ("B" + (n*2) + " ");

}

void fun (int n) {

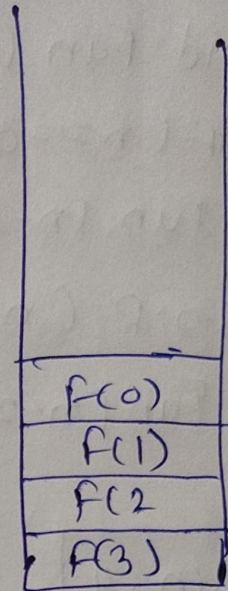
if (n <= 0) return;

S.o.p ("A" + n + " ");

fun (n-1);

S.o.p ("B" + (n*2) + " ");

}



O/p :-

A3

A2

A1

B1

B4

B6

.

Recursion Tree.

Fun(3)

A3

B6

fun(2)

A2

B4

fun(1)

A1

B1

Fun(0)

return

return


```
void fun (int n) {  
    if (n == 0) return;
```

Fun (n-1);

S.O.P (n + "1");

Fun (n-2);

Call: Fun: 4

```
void fun (int n) {  
    if (n == 0) return;
```

```

    fun(n-1);
    S.o.p (n + " ");
    fun(n-2);
}

```

```
void fun (int n) {  
    if (n == 0) return;
```

```

    fun (n-1);
    s.o.p (n+" ");
    fun (n-2);
}

```

```
void fun(int n) {
    if (n == 0) return
```

```

    fun(n-1);
    s.o.p(n+" ");
    fun(n-2);
}

```

```

    }
    void fun (int n) {
        if (n == 0) return
        fun (n-1);
        S.o p ( n + " ");
    }
    fun (n-2);
}

```

٤٤

$$\begin{array}{r} f(1) \\ \hline f(2) \\ \hline f(3) \\ \hline f(4) \end{array}$$

Q.9) void fun (int n) {
 if (n <= 1) {
 S.o.p (n + " ");
 return;
 }

fun (n-1);
 S.o.p (n + " ");
 fun (n-2);

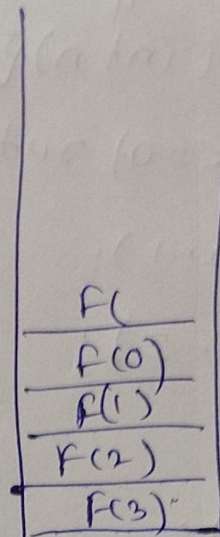
{
 call: fun(3);

→ void fun (int n) {
 if (n <= 1) {
 S.o.p (n + " ");
 return;
 }

{
 fun (n-1);
 S.o.p (n + " ");
 fun (n-2);

{
 → void fun (int n) {
 if (n <= 1) {
 S.o.p (n + " ");
 return;
 }

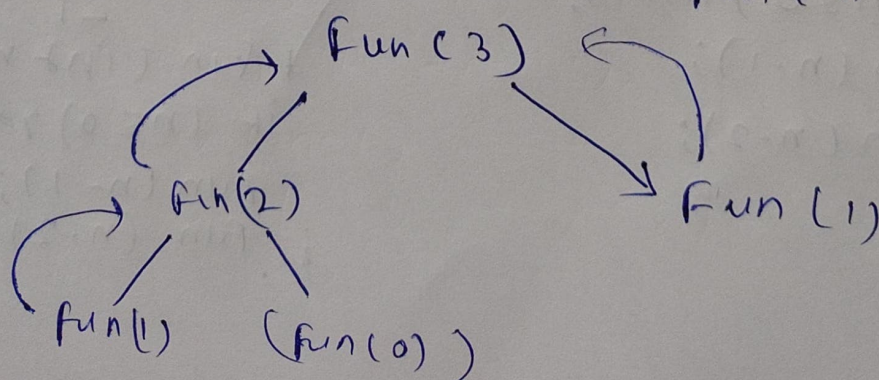
{
 fun (n-1);
 S.o.p (n + " ");
 fun (n-2);
 }



o/p :-
 1
 2
 0
 3
 1

3-2
 void fun (int n) {
 if (n <= 1) {
 S.o.p (n + " ");
 return;
 }
 fun (n-1);
 S.o.p (n + " ");
 fun (n-2); 3-2 = 1
 }

0
 void fun (int n) {
 if (n <= 1) {
 S.o.p (n + " ");
 return;
 }
 fun (n-1);
 S.o.p (n + " ");
 fun (n-2);



Q. 10]

```
void fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);  
    fun (n-2);  
}
```

Call: fun(4);

```
void fun(int n) {  
    if (n <= 0) return;
```

```
    fun (n-1);
```

```
    fun (n-2);
```

```
void fun (int n) {  
    if (n <= 0) return;
```

```
    fun (n-1);  
    fun (n-2);
```

```
fun (int n) {  
    if (n <= 0) return;
```

```
    fun (n-1);  
    fun (n-2);
```

```
fun (int n) {  
    if (n <= 0) return;
```

```
    fun (n-1);
```

```
    fun (n-2);
```

```
void fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);  
    fun (n-2);
```

```
void fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);
```

```
void fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);  
    fun (n-2);
```

```
fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);  
    fun (n-2);
```

```
void fun (int n) {  
    if (n <= 0) return;
```

```
    fun (n-1);  
    fun (n-2);
```

```
fun (int n) {  
    if (n <= 0) return;  
    fun (n-1);  
    fun (n-2);
```


reference
x

4
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}

2.
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}

1
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}

0
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}

0
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}

-1
void fun(int n) {
if (n ≤ 0) return;
fun(n-1);
fun(n-2);
}